
Data Theories: Formalizing the Interpretative Layer between Data and Foundational Ontologies

Applied Ontology
XX(X):2–40
©The Author(s) 2025
Reprints and permission:
sagepub.co.uk/journalsPermissions.nav
DOI: 10.1177/ToBeAssigned
www.sagepub.com/

SAGE

Riley Moher¹ and Michael Gruninger¹

Abstract

Foundational ontologies provide formal, highly expressive accounts of reality. While data-driven applications increasingly rely on these models for semantic interpretation over large-scale datasets, the exact relationship between empirical observations and ontological representation remains opaque and ambiguous. Current approaches either prioritize expressive conceptual modelling—leaving the interpretation of data records implicit—or focus on pragmatic ontology-based data access (OBDA), where lightweight mappings enable querying but obfuscate the underlying ontological interpretations.

To bridge this semantic gap, we introduce *data theories*: a formal framework that explicitly captures how datasets are interpreted with respect to foundational ontologies. Serving as a crucial intermediate layer between abstract conceptual models and operational data analysis, data theories formalize the assumptions, constraints, and transformations required to map raw data records to ontological entities and relations. By making these interpretative commitments explicit, this framework ensures that foundational ontologies are systematically grounded in real-world datasets, facilitating their true operational realization.

We demonstrate the utility of data theories using foundational process ontologies applied to real-world *process mining* datasets. Because process mining relies on reconstructing complex behaviours from raw event logs, it serves as a rigorous evaluation environment for testing the formalization of interpretative decisions. Ultimately, while validated in process-centric scenarios, this framework provides a domain-independent methodology for robust, explicitly defined, ontology-driven data interpretation.

Introduction

The phenomena we observe in reality involve a complex network of interconnected concepts such as space, time, identity, and existence. In data-driven settings, these concepts are often treated intuitively, with implicit ontological commitments adopted as necessary interpretative assumptions. This challenge is compounded by the nature of data itself. Information systems produce representations of reality that are intended to describe real-world phenomena, yet inevitably provide only imperfect approximations. Applied ontology has long sought to address this semantic gap: the missing meaning between formal representations and the phenomena they are intended to capture.

Existing work largely follows two complementary directions. On one hand, foundational ontologies (Guizzardi, Botti Benevides, Fonseca, Porello, Almeida & Prince Sales, 2022; Otte, Beverley & Ruttenberg, 2022; Grüninger, Ru & Thai, 2022; Borgo, Ferrario, Gangemi, Guarino, Masolo, Porello, Sanfilippo & Vieu, 2022a) provide domain-independent conceptualizations of reality, while expressive domain ontologies extend these foundations to model application domains with considerable semantic depth (Ondraszek, Waitelonis, Keller, Niessner, Jacyszyn & Sack, 2025; El Ghosh, Naja, Abdulrab & Khalil, 2025; Brandão & Loureiro, 2020). Although these domain ontologies are developed with reference to real-world datasets, the data primarily serves as a source of domain knowledge; the process by which individual records are interpreted as statements in the language of the ontology is left implicit. The emphasis is therefore on conceptual representation rather than operationalizing ontology-based interpretations over data.

On the other hand, pragmatic frameworks such as ontology-based data access (OBDA) (Xiao, Calvanese, Kontchakov, Lembo, Poggi, Rosati & Zakharyashev, 2018), ontology-based data integration (OBDI) (Gagnon, 2007), and ontology-mediated query answering (OMQA) (Bienvenu, 2016) * focus on efficiently relating data to lightweight ontologies through mappings that support querying and integration. These approaches make minimal ontological commitments, with the domain ontology being a lightweight conceptual layer rather than an extension of some foundational ontology. Consequently, while they provide a formalization of *mapping* data, they do not explicitly characterize how data should be *interpreted* with respect to an underlying foundational ontology.

¹University of Toronto

Corresponding author:

Riley Moher, riley.moher@mail.utoronto.ca

*We will use “OBDA” broadly in this paper to refer to ontology-based approaches for connecting ontologies with existing data sources, including OBDI and OMQA. While these approaches have important technical distinctions, they share the relevant characteristic for our discussion of providing ontology-mediated mapping of data source(s) to lightweight ontologies.

This work is motivated by the challenge of achieving *operational realization*: enabling expressive ontologies to support reasoning over large datasets in real, application-driven scenarios. In previous work, we introduced an architectural perspective on operational realization (Moher & Gruninger, 2025b), highlighting the practical requirements for integrating ontologies with data-intensive applications. However, a fundamental theoretical question remains: how should the relationship between an ontology and the datasets over which it reasons be formally characterized? In particular, operational use of an ontology requires making explicit the interpretative assumptions through which raw data records are interpreted relative to some ontology. This paper addresses this question through the introduction of *data theories*, providing a formal account of the often implicit interpretative layer between foundational ontologies and empirical datasets.

For validation and demonstration of data theories, we focus on the ontology of processes. While this framework is applicable to foundational ontologies generally, processes provide a particularly appropriate setting due to both their ontological significance and the availability of rich empirical data. From an ontological perspective, processes occupy a highly general and domain-independent position in many foundational ontologies, where the distinction between processes (or occurrents) and continuants (or endurants) represents one of the most fundamental ontological commitments. From an empirical perspective, the *process mining* discipline (Van Der Aalst, Adriansyah, De Medeiros, Arcieri, Baier, Blickle, Bose, Van Den Brand, Brandtjen, Buijs & others, 2012; Van Der Aalst, 2012; dos Santos Garcia, Meinheim, Junior, Dallagassa, Sato, Carvalho, Santos & Scalabrin, 2019) provides a mature collection of real-world process datasets and analytical scenarios concerned with interpreting and reasoning about processes from observations sourced from information systems.

These process mining scenarios expose many of the central interpretative challenges addressed by this work: determining what constitutes an event, how events relate to participating entities, how temporal structure is represented, and how observation records correspond to the underlying realities of a particular process. Thus, process mining serves not as the target application domain of the framework, but as a rigorous and data-rich environment in which to examine the operational grounding of foundational process ontologies. The resulting methodology is intended to generalize beyond process mining to other domains where data must be interpreted through expressive ontological commitments.

Contributions. This paper makes the following research contributions:

1. A formal definition of *data theories* - a set of logical theories and mappings that formally encodes the interpretation of some structured dataset(s) relative to a foundational and domain ontology.
2. Formal characterizations of existing methodologies within the data theory framework - including ontology-based data access (OBDA), and ontology-mediated question answering (OMQA). We demonstrate how these methods are a restricted form of data theory, and how data theories in general are capable of

expressing a much greater range of ontological commitments and interpretative decisions.

3. Detailed examples of data theories in real-world settings drawn from the process mining community in the finance and IT domains, leveraging the process specification language (PSL) (Grüninger, 2004) as our chosen foundational process ontology.
4. An open-source implementation library to analyze datasets using data theories, with a focus on process mining tasks and leveraging PSL. The implementation is also intentionally extendable to a wide range of datasets and foundational ontologies.

The remainder of this paper is structured as follows. Firstly, we frame the semantic gap between process data and ontological interpretation as the key opportunity for data theories. The following section then examines processes as a grounded focal domain for data theories, and provides background and motivation for process mining in this context. The next section then presents the core methodological contribution by formalizing data theories as the interpretive layer required for operational realization. We also then elaborate on how data theories differ from other ontology and data integration and mapping techniques, providing specific formalizations of how OMQA and related techniques (Bienvenu, 2016; Xiao, Calvanese, Kontchakov, Lembo, Poggi, Rosati & Zakharyashev, 2018; Gagnon, 2007) are realized as data theories. The next section then demonstrates how data theories are applied in concrete analytical settings, including a case study in an IT ticket support system, and one for financial loan applications. We then present the implementation choices and practical considerations for implementation of data theories and how we combine several logical reasoners to achieve tractable reasoning with data theories over large real-world datasets. Finally, we reflect on some of the limitations of the framework and notable future directions, before finally providing concluding remarks.

Ontologies, Data and Interpretation: Exploring The Semantic Gap

The central challenge addressed in this work is the *semantic gap*: the difference between the information represented in a dataset and the interpretation that can be given to that information in a particular context. The specific terminology of a “semantic gap” can be traced back to the early 2000s, where Smeulders and colleagues defined the concept in the context of content-based image retrieval (Smeulders, Worring, Santini, Gupta & Jain, 2002):

The semantic gap is the lack of coincidence between the information that one can extract from the visual data and the interpretation that the same data have for a user in a given situation.

Although originally formulated for visual data, the underlying problem is more general: representations of reality do not contain their own interpretation, and understanding what a representation means requires assumptions about how it relates to the situation

being analysed. In applied ontology, foundational ontologies provide an important means of addressing this problem by offering rigorous conceptualizations through which observations can be interpreted. However, they do not eliminate the semantic gap. Rather, they raise a further question: given an ontology and some observations, how should those observations be interpreted within that ontology, and what determines which of several possible interpretations is appropriate?

This issue is apparent even in curated ontology evaluation scenarios. The FOUST project, for example, evaluates and compares the ontological commitments of several upper ontologies (Borgo, Galton & Kutz, 2022b) using curated scenario problems. One such scenario describes the following situation:

There is a four-legged table made of wood. Some time later, a leg of the table is replaced. Even later, the table is demolished so it ceases to exist although the wood is still there after the demolition

Each ontology makes distinct ontological choices about how to model this situation, and there are several valid alternative interpretations that could have been chosen for these ontologies. As specifically acknowledged in the case of DOLCE's handling of this case (Borgo, Ferrario, Gangemi, Guarino, Masolo, Porello, Sanfilippo & Vieu, 2022a), the given situation had (at least) two alternatives: a representation where the table and its legs are considered artifacts, or a representation where the roles served by of the pieces of wood in the functioning of the table are reified entities. Ultimately, the chosen interpretation for DOLCE in this case takes the former approach of an artifact-based representation. The chosen interpretation for UFO (Guizzardi, Botti Benevides, Fonseca, Porello, Almeida & Prince Sales, 2022) however, is directly connected to roles, and the constitution relationship in describing portions of wood relates the physical matter to the distinct role each leg plays in supporting the table. This raises some interesting questions and considerations for how the situation 'ought to' be modelled or for what purpose. Clearly, several conceptual modelling decision points exist between ontological commitments, conceptual fidelity or granularity levels. Consider in DOLCE's representation where the legs are all related to the table through the same constitution relation, while in the case of UFO there are distinct roles for each of the four legs (*LeftFront*, *RightRear*, etc). UFO takes an assumption that *any* valid table must have exactly four legs and each leg must be separately identifiable, inherently forbidding a table with only three legs, or a single post-like leg, for example. This is despite the original statement not explicitly saying the *only* kind of table that may exist is one with four legs. Because of the nature of the initial description being inherently ambiguous as it is given in natural language, there is a high degree of permissibility here in conceptual modelling choices. The semantic gap in this case arises from this mismatch between the inherent precision in the foundational ontology and the loose natural language form of the given data. An especially significant issue to consider here is the task at hand, or the given situation for which the natural language data is being mapped into the language of the ontologies. In this case, the application at hand is a generic representational problem with the implicit task of demonstrating the ontology. This will naturally draw out more verbose representations, and does not have any regard for pragmatics or scalability. Contrast this

to a more grounded and outcome-driven approach where a definite and precise answer must be given. For instance, demonstrating that the mass of the initial table is equal to the mass of the resultant wood [†]. In this context, the utility of the role-based approach is less clear and the conceptual modelling decisions points become more straightforward, albeit still with some level of arbitrage in how the concepts in the ontology are mapped to the situation.

Perfect fidelity to reality is rarely the goal of data-driven applications that could nevertheless benefit from structured representations of knowledge. Practitioners routinely make deliberate ontological compromises—simplifying temporal constraints, flattening complex class hierarchies, or abstracting away distinctions that are unnecessary for a particular task—because a representation that is “good enough” often provides greater computational tractability, analytical utility, or compatibility with existing data governance practices.

Foundational ontologies provide a principled basis for semantic interoperability through rigorous conceptualizations of reality, but methods for applying these expressive ontologies to operational data remain comparatively underdeveloped. Much of the applied ontology literature emphasizes axiomatization, conceptual modelling, and logical analysis, treating ontologies primarily as representational artefacts rather than operational schema for reasoning over heterogeneous, large-scale data. As a consequence, practitioners are provided with expressive conceptual models but comparatively little guidance on how existing enterprise data should be interpreted with respect to those models.

Consider an application of UFO to the domain of criminal law (El Ghosh, Naja, Abdulrab & Khalil, 2025), where the foundational ontology is specialized to represent concepts such as judges, offenders, and legal powers. A similar methodology is adopted in an application of BFO to sports science (Ondraszek, Waitelonis, Keller, Niessner, Jacyszyn & Sack, 2025). In both cases, the primary contribution lies in constructing a semantically rigorous domain ontology, with validation consisting of querying example datasets expressed directly in the ontology’s vocabulary. Such evaluations demonstrate that the resulting ontology behaves consistently with its intended conceptualization, but they leave open an important methodological question: how should records from existing information systems be interpreted as instances of the ontology in the first place? The data primarily serves as a reference point for developing and demonstrating the conceptual model, while the interpretative relationship between operational data and the ontology remains implicit.

A different perspective is taken in (Grüninger, Shapiro, Fox & Weppner, 2010), where the authors apply the Process Specification Language (PSL) (Grüninger, 2004) to manufacturing systems with RFID tags. Rather than reasoning over manually constructed ontological instances, the authors use an ad-hoc software mapping to translate the

[†]Of course, this is assuming the replacement leg will not have an altered mass, and the process of replacement and demolition does not add or subtract any weight, which are assumptions which may or may not be represented with the ontology

exported RFID logs into logical assertions expressed in the vocabulary of PSL. This represents a significant step toward the operational use of expressive foundational ontologies over real-world data. However, the semantic interpretation embodied by this translation exists solely as procedural software. The assumptions that determine how observations in the source data correspond to PSL concepts are embedded within the implementation, making them difficult to analyse, verify, compare, or reason about independently of the translation program itself.

Taken together, these works illustrate complementary aspects of the same methodological gap. Conceptual modelling approaches excel at constructing expressive ontological representations, but typically leave the interpretation of operational data implicit. Operational implementations provide mechanisms for transforming data into ontological assertions, but the semantics of those transformations remain embedded procedurally rather than represented explicitly as logical artefacts. In both cases, the interpretative relationship between empirical data and a foundational ontology is assumed rather than formally characterized.

This gap becomes particularly significant in enterprise settings. Traditional conceptual modelling aims to construct well-founded reference models against which information systems can be designed or evaluated. In practice, however, replacing existing data models with ontologically grounded schemas is rarely feasible. Legacy systems, organizational constraints, and heterogeneous data sources mean that analysts routinely work with imperfect representations rather than idealized conceptual models.

Bridging the semantic gap therefore requires complementing upstream conceptual modelling with a formal account of downstream data interpretation. If applied ontology seeks broad practical impact, it must meet organizations where they already operate: not only when information systems are designed, but when existing data is interpreted. Conceptual modelling remains essential for designing well-founded information systems, but opportunities to redesign enterprise schema are relatively infrequent and often constrained by legacy infrastructure, organizational processes, and cost. In contrast, existing data is interpreted and analyzed continuously by analysts, domain experts, and decision-makers who must work within these constraints. Providing ontological support at the point of data interpretation therefore has the potential to reach a far broader set of practical applications while complementing, rather than replacing, traditional conceptual modelling.

Another approach to this issue is the integration of data and ontological perspectives through pragmatic and modular frameworks that establish explicit connections between data sources and ontological models, namely ontology-based data access (OBDA), ontology-based data integration (OBDI), and related approaches such as ontology-mediated query answering (OMQA) (Gagnon, 2007; Lenzerini, 2011; Xiao, Calvanese, Kontchakov, Lembo, Poggi, Rosati & Zakharyashev, 2018). At a basic level, OBDA provides a mechanism for exposing heterogeneous data sources through an ontology-mediated interface, allowing users to query data using a conceptual vocabulary rather than the underlying storage schema. This has proven valuable for integrating existing information systems with semantic technologies without requiring costly restructuring of the underlying data infrastructure.

However, the primary objective of OBDA is to facilitate access to and integration of existing data sources rather than to formally characterize the interpretative assumptions that connect those data sources to an ontology. The mappings in OBDA specify how data is transformed into ontological assertions, but the semantic decisions underlying those mappings are generally treated as fixed implementation choices. For example, whether a particular database record corresponds to an entity, event, relation, or state in the domain is determined during the construction of the mapping but is not itself represented as an explicit logical artifact. Consequently, OBDA provides an effective mechanism for querying over an ontology-mediated view of data, but it does not address the broader question of how empirical data should be interpreted with respect to foundational ontological commitments.

Processes in Focus

To illustrate the need for explicit data interpretation in a concrete domain, we focus on the mid-level process domain. Processes provide a particularly useful setting for examining the semantic gap because their analysis requires reconciling flexible static data sources with complex, temporal behaviour. Understanding such behaviour from the data produced by operational information systems therefore involves interpretative assumptions about how recorded data relate to the underlying process. This challenge is particularly evident in organizational settings, where processes describe how activities, resources, and other entities interact over time to accomplish organizational objectives. Such processes have long been studied through Business Process Management (BPM), a field concerned with their modelling, analysis, and improvement (Van der Aalst, 2013). Traditionally, BPM relied on expert-driven approaches such as Business Process Re-engineering (Grover & Malhotra, 1997), where process models were constructed from domain knowledge and stakeholder input. The increasing availability of enterprise information systems has enabled a complementary data-driven paradigm: *process mining*, which uses event data generated by operational systems to discover, analyze, and monitor process behaviour from observed executions (Van der Aalst, 2013).

Event logs, the primary data source of process mining, encode processes as collections of timestamped observations, but typically provide limited semantic structure (Gunther & Verbeek, 2014; Accorsi, Di Ciccio, Jayarathna & Verbeek, 2022). Important notions such as event identity, participation, causality, temporal relations, and relationships between multiple objects are often implicit. Recent extensions such as object-centric event logs (Berti, Koren, Adams, Park, Knopp, Graves, Rafiei, Liß, Unterberg, Zhang & others, 2024) address some limitations of traditional event log representations, but the fundamental challenge remains: the data representation does not explicitly capture the ontological commitments required for deeper process reasoning.

In this work, we focus specifically on discrete event logs as the input representation for process analysis. We do not address the broader challenge of constructing event logs from heterogeneous or unstructured sources, such as sensor streams, text descriptions, or external data integration scenarios. While ontologies may support these tasks through approaches such as ontology-based data access (OBDA) (Xiao, Calvanese, Kontchakov,

Lembo, Poggi, Rosati & Zakharyashev, 2018), semantic parsing (Kamath & Das, 2018), or data warehousing (González López de Murillas, Reijers & Van Der Aalst, 2019), these approaches focus on creating or integrating process data rather than reasoning over an established process representation.

Similarly, our focus differs from ontology-based approaches in automated planning and robotics, where process knowledge is used to generate future actions or control behaviour (Marrella, 2019; Olivares-Alarcos, Beßler, Khamis, Goncalves, Habib, Bermejo-Alonso, Barreto, Diab, Rosell, Quintas & others, 2019). Instead, we consider an analytical setting in which process executions have already occurred and the goal is to understand and reason over the resulting observations.

This distinction is important because process mining relies on more than the information explicitly contained in an event log. Analysts routinely apply implicit knowledge about process structure, temporal behaviour, participation, and domain context when interpreting observed executions. Our objective is to make this knowledge explicit and formally represent how it is applied to event data.

Therefore, the goal of this work is not to construct process data or generate new executions, but to provide a formal basis for interpreting existing observations. Process mining provides a particularly suitable environment for this investigation because its analytical tasks require reasoning over complex temporal and behavioural patterns while operating on empirical data generated from real processes.

Event Logs

The central artifact in process mining is the *event log*. It is a record of events that have occurred over a fixed period of time, as captured by some process-aware information system(s) (PAIS). The recorded events within an event log are usually stratified at the top level by a process (so one event log pertains to one process), and within the log by *cases*. While not explicitly identified in this manner, cases can be understood as identifiers for the instances or occurrences of the log's top-level process. The set of unique events associated with a given trace comprises a trace, an ordered sequence of *events*, with each event being described by some set of attributes. At a minimum, event attributes include an **activity name** (or event label), describing the task performed, and a **timestamp** recording when the event occurred. Additional attributes, such as the **resource** the actor or system performing the activity, are also commonly included. The widely-utilized format for event logs is the extensible event stream (XES) format (Gunther & Verbeek, 2014), an XML-based format which is intentionally very flexible, allowing a wide range of events to be captured.

The XES standard definition for an event is “an atomic granule of activity that has been observed” - a conceptually broad definition that is reflected in the challenges of interpreting and analyzing event logs. While the ontological flexibility of events enables process mining to be applied across a diverse set of domains, it also introduces significant semantic heterogeneity. Events, as captured in XES logs, depict many diverse process concepts, including status values, object states, or occurrences. Consequentially, interpretation and disambiguation of such data requires domain knowledge to bring out

the intended semantics. For instance, consider real-world event labels drawn from a dutch municipality’s building permit approval process (van Dongen, 2015), depicted in Table 1.

Table 1. Building permit log example

Activity	Timestamp
Enter senddate send procedure confirmation	09:42:54
Send procedure confirmation	09:42:54
No permit needed or only notification needed	15:09:51
Treat subcases completeness	15:10:44

While the assumption behind event logs is that all recorded events can be treated as a single ordered sequence of homogeneous events, real-world event labels reveals substantial semantic heterogeneity. In the example shown in Table 1, the first two events—“Enter senddate send procedure confirmation” and “Send procedure confirmation”—are not merely sequential occurrences but appear to be semantically related, with the former describing a preparatory or meta-action leading to the latter. Yet, within the log, this relationship is collapsed into a simple ordering without any explicit linkage or dependency. Moreover, they share an identical timestamp, suggesting that rather than these being truly concurrent activities, both events may be the result of a common action by the information system(s). This introduces ambiguity in interpretation of the data and opens up several different methods for process mining practitioners to analyze and ‘clean’ the data in accordance with their analysis goals.

Additionally, the contrasting activity labels “Send procedure confirmation” and “No permit needed or only notification needed” illustrate the coexistence of distinct semantic types within the same representational layer: one expresses an action performed within the process, while the other designates a state or decision outcome. Despite their fundamentally different roles—one eventive, the other stative—both are treated equivalently as atomic entries in the log. Flattening of process information into sequential and eventive atoms introduces significant limitations for casual and mereological relationships.

Finally, the event “Treat subcases completeness” introduces an additional dimension of complexity, suggesting a mereological relation: the handling of subcases as parts of a larger case structure. This notion of part–whole dependency, inherent in many administrative or regulatory processes, cannot be adequately represented or reasoned about within the conventional event log formalism. Process mining tools, which operate on this flattened representation, thus lack the means to distinguish between event relations, event types, and mereological structures, all of which carry significant semantic implications for process understanding and analysis.

In addition to the issues introduced by flexibility in events and their labels, XES also defines extensions to add additional columnar data for each event. The XES lifecycle extension defines an additional description for events called transitions, which have a standard set of labels such as “start,” “complete,” “schedule,” and “abort.” However, many real-world logs omit lifecycle tags entirely, or use ad-hoc labels, such as in the

BPIC 2013 dataset (Steeman, 2013), where simple event labels like “Accepted” and “Queued” are supplemented with lifecycle transitions such as “Awaiting Assignment.” Without ontological grounding or standardization, this variability makes it difficult to reconcile data across different datasets and domains. While efforts like object-centric event logs (OCEL) (Ghahfarokhi, Park, Berti & van der Aalst, 2021) attempt to alleviate some of these issues by better structuring logs around objects like orders, shipments, or diagnoses, semantic heterogeneity of events and informality in data handling remain critical, unaddressed issues.

Process Mining Analysis

In addition to the challenges and opportunity given by event logs in process mining and the ways in which they (fail to) encode process information, the downstream analysis tasks of process mining introduce novel and promising opportunities for process ontology evaluation. In process mining, the process model serves as the principal artifact through which behaviour captured in event logs is interpreted, analyzed, and improved. This centrality of the model is reflected in the three primary categories of process mining techniques: process discovery, conformance checking, and process enhancement (Van Der Aalst, 2012).

Process discovery algorithms automatically construct explicit process models—typically represented in formalisms like Petri nets or BPMN—from event logs. These models aim to capture the control-flow logic observed in the recorded behaviour (Van Der Aalst, 2012). Discovery algorithms inherently negotiate a trade-off between competing quality dimensions, such as **completeness** (capturing all observed behaviour), **precision** (avoiding unobserved behaviour), and **simplicity** (generalizing from noise or exceptions). However, a persistent challenge, even for recent neural network approaches (Sommers, Menkovski & Fahland, 2021, 2023), is the generation of highly complex and uninterpretable “spaghetti models,” which often arise when raw, unfiltered event logs are used as direct input.

To mitigate this, practitioners frequently pre-process or abstract the event log, guided by domain knowledge and specific analysis goals. These manual interventions, however, embed implicit or covert ontological commitments. This is evident in analyses, such as that of loan application data (Bautista, Wangikar & Akbar, 2012), where practitioners manually abstract activity labels to enhance model interpretability. For instance, observed sequences like ABCD, ACBD, and BACD—where a set of activities $\{A, B, C\}$ always co-occur in an arbitrary order before a subsequent activity D—is identified as “redundant.” This entire set $\{A, B, C\}$ is then abstracted into a single, representative activity label (e.g., relabelling all instances as A). While this pragmatic step addresses the “spaghetti model” problem, it obfuscates underlying ontological assumptions, namely by conflating **process mereology** with simple behavioural redundancy. This common, pragmatic challenge thus presents a clear opportunity for formalization using process ontologies. Additionally, the focus of process discovery on creating control-flow process models also misses important nuances. Focusing specifically on control flow misses elements like resource allocation, or restrictions on processes above the individual execution level,

so things like “The maximum amount of debt issued in a month cannot exceed \$3M”, for example. With typical process models like petri nets, they serve as a description of the process only in terms of a single occurrence, not in terms of abstracting or restricting multiple occurrences over time or other dimensions like responsible parties, etc. From an ontological standpoint, this reduces the process description to only a single aspect, and further exacerbates issues of semantic heterogeneity of events, as previously mentioned.

Conformance checking focuses on comparing observed behaviour in the event log against some existing process model(s). The objective is to identify discrepancies between the model and the actual executions—whether deviations occur in terms of missing, reordered, or unexpected activities. This form of analysis can quantify compliance, help to explain anomalous behaviour, and provide additional information to support process model refinement and auditing. Conformance checking thereby transforms the event log into an evidential layer for assessing model validity and operational adherence. However, conformance checking is only as reliable as the sum of its parts. Just as in process discovery, the input event log will often need to be pre-processed in order to align with the conceptual and granular abstractions of the reference model. This alignment, much like the pre-processing in process discovery, obfuscates implicit ontological choices.

The central challenge is that traditional conformance checking operates at the syntactic level of activity labels, lacking a semantic understanding of what those labels represent. A reference model, for instance, may specify a high-level activity like “Assess Loan Risk”. The event log, however, may contain a sequence of more granular, concrete activities such as Check Credit Score, Verify Income, and Review Debts. A standard conformance checker, such as one using token-based replay on a Petri net, would incorrectly register these granular activities as deviations (i.e., unexpected events), as their labels do not syntactically match the single expected label of “Assess Loan Risk”.

This apparent non-conformance is not an operational failure but a semantic mismatch, rooted in a failure to formally represent the process ontology. The problem is a misalignment of granularity, which can only be resolved by appealing to process mereology (where Check Credit Score is a part of Assess Loan Risk) or taxonomy (where, for example, a logged Urgent Triage is a specific type of the modelled Triage activity).

Conformance checking, therefore, provides a powerful and practical testbed for evaluating an ontology’s utility. An effective process ontology would supply the necessary axioms to formally relate the granular log events to the abstract model activities. This allows the conformance checker to move beyond simple string matching and verify semantic conformance, thereby correctly identifying true deviations while properly grounding the observed, granular behaviour in the normative model.

Furthermore, this problem is compounded by the limited scope of the models themselves. Just as in discovery, the reference models used for conformance are often control-flow-centric. They may confirm if the prescribed sequence of activities occurred, but they typically cannot validate if the execution was compliant with deeper, multi-dimensional business constraints—such as the data-centric rule that “The maximum amount of debt issued in a month cannot exceed \$3M.” Conformance checking thus

inherits this narrow focus. Evaluating the conformance of data, resources, or temporal constraints (e.g., deadlines) requires a reference model (and a supporting ontology) that explicitly formalizes these aspects in the first place, presenting another clear and critical opportunity for ontological evaluation.

Process enhancement builds upon discovery and conformance results by using event data to refine or extend existing process models. Enhancement can take multiple forms, including enriching models with performance information (e.g., throughput times or resource utilization), integrating contextual attributes, or adapting control-flow structures to reflect evolving organizational behaviour. The overarching aim is to leverage the empirical grounding provided by event logs to improve both the descriptive and prescriptive quality of the model.

Together, these three categories form the methodological core of process mining. However, they all rely on the foundational assumption that the event log provides a semantically consistent and unambiguous record of process execution—a premise that becomes increasingly problematic when the log’s heterogeneity, as discussed earlier, obscures distinctions between actions, states, and relational dependencies within the recorded behaviour.

This shift towards object-centricity represents a significant step towards a more ontologically sound representation of processes, as it moves from a flat, sequential view to a relational view of process execution. This richer data model provides a more faithful representation of reality and, consequently, a more suitable foundation for applying and evaluating the expressive power of foundational process ontologies.

Process Mining Datasets and Benchmarks

Finally, the *BPI Challenge* (BPIC) datasets (Lopes & Ferreira, 2019; Thiede, Fuerstenau & Bezerra Barquet, 2018) serve as a de-facto standard for benchmarking in the field. However, the application of these real-world logs inherently requires a “data theory”—an interpretation of the data in the context of some domain process knowledge. We leverage this high-volume, empirically grounded environment to assess the operational readiness of process ontologies, using the datasets not merely as inputs for application, but as a rigorous testing ground for robustness and validity.

This reliance on interpretation is evident in standard process mining tasks, where implicit domain knowledge is frequently used to structure the data. For example, in the foundational work on neural network-based prediction by Tax et al. (Tax, Verenich, La Rosa & Dumas, 2017), the large BPIC2012 log is decomposed into three distinct sub-processes: application tracking, work item state tracking, and offer state tracking. This decomposition necessarily relies on the interpretation of event labels against external domain knowledge. Furthermore, the interpretation of event labels is done in co-ordination with an implicit domain hierarchy of the process. Additionally, to facilitate next-event prediction, the authors restrict the log to the work item sub-process and only include “complete” events (as marked by their lifecycle transitions). This exact same pre-processing and interpretive assumption regime is applied in many foundational predictive process monitoring works (Bukhsh, Saeed & Dijkman, 2021??), highlighting

the significant reliance on implicit data theories necessary required to render these benchmarks usable.

This standard practice raises critical methodological questions. If high-performance models require extensive yet undocumented data processing—such as discarding “schedule” or “start” events—this raises serious concerns for not only what is being lost in interpretation, but what is being discarded or disregarded entirely. It is counter-intuitive that the timing of a “start” event would be considered irrelevant for predicting the timing of any corresponding “end” events, for example. Furthermore, the lack of a formal justification for these decisions poses a scalability risk: as datasets become more complex and noisy, this lack of standardization makes it difficult to distinguish noise and meaningful observations in the data.

Ultimately, employing process mining techniques in the real world necessitates the interpretation of this messy data to fit the rigid and inflexible requirements of process mining techniques, including machine learning architectures. While ad-hoc simplifications may yield pragmatic results, they highlight a dependency on implicit, “black-box” interpretations of the domain - the data theory is opaque. In this work, we do not seek to leverage this dependency solely as a flaw to be corrected, but as a metric for evaluation. The necessity of explicating this domain knowledge creates opportune conditions for evaluating the operational readiness of a process ontology, testing its ability to formally support the data theory that current techniques assume implicitly.

Process Mining Ontology

While the process mining community has increasingly sought to more precisely define and understand event log data, existing approaches largely focus on representation and access rather than rigorous reasoning or operational realization. We categorize these efforts into three streams: graph-based data access, declarative constraints, and formal ontologies.

Graph-Based and Data Access Approaches Several approaches attempt to bridge the gap between raw data and process models using graph structures, yet they often lack explicit semantic commitment. Event Knowledge Graphs (EKGs) (Fahland, 2022) allow for multi-perspective process analysis (e.g., handling purchase orders vs. shipments) by lifting data into labeled property graphs via Cypher queries. However, as noted in (Swevels, Fahland & Montali, 2023), the mappings in these systems remain implicit within query structures or “semantic headers,” limiting transparency and reuse compared to formal mapping standards like RML (Dimou, Vander Sande, Colpaert, Verborgh, Mannens & Van de Walle, 2014). Crucially, EKGs assume perfect input data quality, offering no formal mechanism to reason about the inconsistencies inherent in real-world event logs.

Similarly, Ontology-Based Data Access (OBDA)—exemplified by Onprom (Calvanese, Kalayci, Montali & Tinella, 2017)—maps data to RDF graphs to enable SPARQL-based retrieval. While useful for abstraction, OBDA in this context is often limited to simple subsumption and cardinality constraints. It functions primarily as a translation layer rather than a reasoning layer, lacking the expressivity to resolve

semantic heterogeneity or model complex temporal dynamics without ad-hoc, external pre-processing.

Declarative and Temporal Logics DECLARE (Di Ciccio, Montali & others, 2022) and its data-aware extensions (Bergami, Maggi, Marrella & Montali, 2021) represent the standard for declarative process modelling, specifying process behaviours as Linear Temporal Logic (LTL) constraints (e.g., “A must eventually follow B”). While having formal semantics, DECLARE and its extensions treat process entities as homogeneous attribute-value pairs, reducing complex domain objects to flattened payloads. This abstraction prevents the explicit modelling of ontological distinctions—such as participation, mereology, or event-state relationships. Furthermore, because DECLARE relies on point-based semantics, integrating interval-based semantics of a foundational ontology is a significant challenge. As with graph-based approaches, these systems act as validators for data that is assumed to be structurally sound, rather than as tools for interpreting semantically ambiguous process data.

Semantic Web and Upper Ontologies Efforts to apply standard ontologies to process mining face a dichotomy of lightweight and heavyweight approaches.

On one end, Semantic Web Ontologies like the Event Ontology (EVO) (Pedrinaci & Domingue, 2007) or BPMO (Norton, Cabral & Nitzsche, 2009) serve as taxonomies for tagging event types like “start,” or “end”. As demonstrated by Katsumi and Grüninger (Katsumi & Grüninger, 2015), these models lack the expressivity to answer basic competency questions about process dynamics. Benevides et al. (Botti Benevides, Bourguet, Guizzardi, Peñaloza & Almeida, 2019) similarly found that standard OWL/Semantic Web constructs are unsuitable for representing the foundational semantics of processes required for robust reasoning.

On the other end, Upper Ontologies (e.g., DOLCE (Borgo, Ferrario, Gangemi, Guarino, Masolo, Porello, Sanfilippo & Vieu, 2022a), UFO (Guizzardi, Botti Benevides, Fonseca, Porello, Almeida & Prince Sales, 2022) and the Process Specification Language (PSL) (Grüninger, 2004)) offer deep, domain-independent axiomatizations. However, this expressivity comes at the cost of conceptual bloat. For instance, modelling a simple “running” event in DOLCE requires a complex set of interrelated concepts of perdurants, accomplishments, and temporal qualities that are difficult to map to a standard event log. While PSL is an ISO standard, its reliance on first-order logic introduces undecidability issues and a lack of practical tooling for handling the noise and heterogeneity of real-world datasets.

Ultimately, a gap remains: lightweight methods lack formality and reasoning power to appropriately handle real-world data, while expressive and heavyweight formalisms lack thoughtful tool support to be applied to process mining at scale.

Data Theories

The preceding discussion argues that a fundamental component of ontology-driven data analysis remains largely implicit: the assumptions that determine how empirical data is interpreted with respect to an ontology. We refer to this collection of interpretative

assumptions as a data theory. A data theory provides an explicit logical characterization of how a particular dataset is understood in terms of the concepts, relations, and commitments of a foundational ontology. It therefore forms the missing semantic layer between a general ontological model and the concrete observations contained in a dataset.

Consider the following interpretative decisions one may be faced when analyzing data:

- In a customer database, does a null value in the *region* column imply missing data, no applicable region, or a higher-level *global* region?
- In an event log, does an event's recorded timestamp refer to the start of an activity, its completion, or the time at which it was recorded by the information system?
- In a healthcare system, does a patient identifier refer to a single individual across different care providers, or only to a patient record within a particular institution?
- In a manufacturing system, does a machine identifier represent a physical machine or a temporary allocation of capacity?

The answers to these questions are not contained within the dataset itself, nor are they directly defined in a domain ontology (if one even exists), or any upper-level foundational ontology. This logic—the specific set of assumptions required to connect more general knowledge to instance-level data—is effectively a third, distinct component. In traditional process mining and data science workflows more broadly, this interpretative component remains largely ad-hoc and informal; at best opaquely encoded in software code or informal documentation. We explicitly formalize this interpretive component as a necessary part of operational realization: the *data theory*.

Furthermore, these interpretative decisions do not have a single correct answer. Given a domain ontology, a raw dataset, and a foundational ontology, two practitioners may come up with alternative interpretations that are equally valid. Thus several data theories may exist for any given dataset. Consider an event log containing the event sequence $A \rightarrow B \rightarrow C$. Two distinct interpretations can inform data theories for the same log:

- **Interpretation A (Abstraction):** Defines a rule that groups A , B , and C into a single complex event instance X , viewing the individual subsumed events as redundant operational detail.
- **Data Interpretation B (Granularity):** Treats A , B , and C as distinct, causally linked events, potentially even inferring additional sub-events.

Both interpretative apply to the same domain and the same data. They are not alternative domain ontologies—which would imply conflicting definitions of the domain's reality—but rather alternative *analytical perspectives*. This separation allows the foundational and domain ontologies to remain general and reusable, while the data theory captures the specific heuristic assumptions required to operationalize the data.

Figure 1 illustrates the intuitive multiplicity of a data theory with respect to a dataset, foundational ontology, and domain ontology(ies).

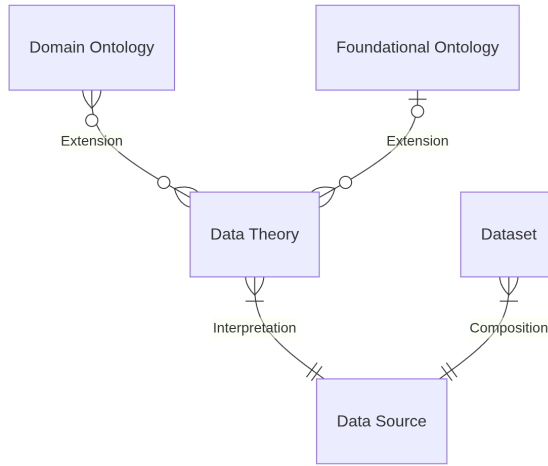


Figure 1. Intuitive Multiplicity of Data Theories

A data source is simply a collection of one or more datasets. To make sense of the information, every dataset requires at least one data theory to interpret it. Conversely, a specific data theory is strictly tied to just one dataset. When building its vocabulary, a data theory might rely on a single foundational ontology for its broad structure, and it can pull from any number of domain ontologies for subject-specific details. We expand on this basic setup in the formal definition of a data theory.

Formalization of Data Theories

Conceptually, a data theory occupies the space between the A-Box (assertional data) and the T-Box (terminological axioms), providing the interpretative layer that connects observations to ontological meaning. Whereas a domain ontology specifies conceptual knowledge that is intended to hold independently of any particular dataset, a data theory captures the dataset-specific assumptions required to apply that knowledge in practice. Although a data theory may extend a domain ontology, its defining characteristic is the explicit representation of the mapping between empirical observations and ontological assertions—a relationship not inherently expressed by domain ontologies themselves.

A data theory captures both the general interpretative assumptions that connect a data source to an ontology and the specific consequences that arise when those assumptions are applied to concrete data. To distinguish these two roles, we characterize a data theory across two levels: the level-zero data theory $\mathcal{T}_{\text{data}}^{(0)}$ and higher-level data theories $\mathcal{T}_{\text{data}}^{(1+)}$. The level-zero theory captures the foundational (or schematic) interpretation of the data source through model-theoretic types, while higher-level theories build on this interpretation to derive additional domain-specific consequences through definable sets.

Level-Zero Data Theory The interpretation of a raw data source with respect to foundational and domain ontologies is first established by $\mathcal{T}_{\text{data}}^{(0)}$. This level encodes structural alignment without introducing contextual domain semantics.

Formally, it consists of conservative extensions specifying partial types over the empty set. Because these types are parameter-free, all formulas define $\emptyset$ -definable sets. This enforces schema-level invariance, producing a structural mapping independent of particular instances observed in the data source.

Let:

- $\mathcal{T}_F$ be a foundational ontology with signature Σ_F ,
- $\mathcal{T}_D$ be a domain ontology with signature Σ_D ,
- Σ_S be a finite relational and functional signature,
- $\mathcal{D}_{\text{raw}}$ be a finite Σ_S -structure containing the source data.

Let $\mathcal{L}_S$ be a designated logical language over Σ_S (e.g., the existential conjunctive fragment of First-Order Logic or full First-Order Logic). A level-zero data theory $\mathcal{T}_{\text{data}}^{(0)}$ consists of logical dependencies of the form:

$$\forall \mathbf{x} \left(\Phi_S(\mathbf{x}) \rightarrow \Psi_F(\mathbf{x}, f(\mathbf{x})) \right)$$

where Φ_S is a well-formed formula in $\mathcal{L}_S$ specifying a partial type over $\emptyset$, and Ψ_F is a conservative definition over Σ_F . The function $f(\mathbf{x})$ provides a globally scoped Skolem term that denotes the ontology-level individual corresponding to the source-level object described by $\mathbf{x}$.

The result of the application of the level-zero data theory is a set of data assertions $\mathcal{A}_D$. These assertions represent the encoding of the data in the language of the foundational and domain ontologies, and are defined as the logical consequences of realizing these types from the raw data source $\mathcal{D}_{\text{raw}}$:

$$\mathcal{A}_D = \left\{ \phi(\mathbf{c}) \mid \mathcal{D}_{\text{raw}} \cup \mathcal{T}_{\text{data}}^{(0)} \cup \mathcal{T}_F \cup \mathcal{T}_D \models \phi(\mathbf{c}) \right\}$$

Note here that $\mathcal{A}_D$ is defined for a *consistent* combined theory. The use of the data theory in reasoning and handling consistencies is elaborated on in the following subsection.

To concretely demonstrate a level-zero data theory, Consider an example based on tracking the status of loan applications. Here $\mathcal{D}_{\text{raw}}$ consists of a relational table, `LoanLog`:

ApplicationID	Timestamp	Status
A101	2026-04-09 09:00	SUBMITTED
A101	2026-04-12 10:15	ACCEPTED

The level-zero data theory provides an interpretation of the data schema with respect to some foundational and/or domain ontologies. In this example we define very lightweight foundational and domain ontologies where `LoanApplication` $\in \Sigma_D$, `Endurant` $\in \Sigma_F$,

and the axiom $(\text{LoanApplication} \sqsubseteq \text{Endurant}) \in \mathcal{T}_D$. The level-zero data theory may contain the following definition to interpret arbitrary rows in the raw data source:

$$\begin{aligned} \forall a, t, s \quad & (\text{LoanLog}(a, t, s) \equiv \\ & \text{Perdurant}(f_{\text{event}}(a, t, s)) \wedge \\ & \text{LoanApplication}(f_{\text{app}}(a)) \wedge \\ & \text{hasTime}(f_{\text{event}}(a, t, s), f_{\text{time}}(t)) \wedge \\ & \text{participatesIn}(f_{\text{app}}(a), f_{\text{event}}(a, t, s))) \end{aligned}$$

Here $\Phi(\mathbf{x})$ is a simple query of the entire table; expressed in SQL terms as a simple select all: `SELECT * FROM LoanLog`, and corresponding to the literal $\text{LoanLog}(a, t, s)$ in the definition. The result of this level zero data theory is the set of logical literals $\mathcal{A}_D$:

$$\begin{aligned} \mathcal{A}_D = \{ & \text{LoanApplication}(a_{101}), \text{Perdurant}(e_1), \\ & \text{participatesIn}(a_{101}, e_1), \\ & \dots \\ & \text{participatesIn}(a_{101}, e_2) \} \end{aligned}$$

This initial set of data assertions would also include any other literals closed under entailment in $\mathcal{T}_F \cup \mathcal{T}_D$, which may introduce additional individuals not produced by the skolem terms of $\mathcal{T}_{\text{data}}^0$ to represent unknown information which exists outside of the data.

Higher-Level Data Theories A higher-level data theory $\mathcal{T}_{\text{data}}^{(1+)}$ moves beyond structural mappings to specify model-theoretic types over parameter sets.

Let $C = \text{const}(\mathcal{D}_{\text{raw}})$ be the active domain of constants. For a specific tuple $\mathbf{c} \in C$, $\mathcal{T}_{\text{data}}^{(1+)}$ specifies the type of $\mathbf{c}$ over the entire parameter set C , denoted $tp(\mathbf{c}/C)$.

While specifying types over C represents the axiomatic intent, the operational consequence of $\mathcal{T}_{\text{data}}^{(1+)}$ is the generation of C -definable sets. By incorporating specific constants from the data source, the theory focuses on the interpretation of highly specific subsets of the data source.

A higher-level data theory is a set of logical sentences constructed over a vocabulary Σ_{DT} that satisfies:

$$\Sigma_{\text{DT}} \cap (\Sigma_D \cup C) \neq \emptyset \quad \text{or} \quad \Sigma_{\text{DT}} \setminus \Sigma_F \neq \emptyset$$

By explicitly requiring the domain vocabulary (Σ_D) and the constants of the data source (C), $\mathcal{T}_{\text{data}}^{(1+)}$ axiomatizes how specific constants are interpreted together and in the context of the domain theory.

In a process context, this may include elements such as event orderings, role participation, and process mereology. It formalizes specific, domain-informed properties onto the data instances, effectively embedding individual data points into a more complete domain-specific context via the evaluation of these C -definable sets.

To illustrate a higher-level data theory, we build upon the `LoanLog` example. Suppose the domain ontology contains specific event subclasses and states, such as an `ApprovalEvent`. A higher-level data theory leverages the specific constant "ACCEPTED" from the active domain C to impose domain-specific semantics:

$$\begin{aligned} \forall a, t \left(\text{LoanLog}(a, t, \text{"ACCEPTED"}) \rightarrow \right. \\ \left. \text{ApprovalEvent}(f_{\text{event}}(a, t, \text{"ACCEPTED"})) \wedge \right. \\ \left. \text{hasStatus}(f_{\text{app}}(a), \text{ApprovedState}) \right) \end{aligned}$$

Rather than applying a generic schema mapping, this rule generates a C -definable set. It isolates tuples containing the parameter "ACCEPTED" and maps them to specific ontology concepts, asserting that this specific event is an `ApprovalEvent` and that the application has an `ApprovedState`. Applying this to the tuple $c = (\text{A101}, 2026-04-12\ 10:15, \text{ACCEPTED})$ yields precise domain assertions that would not be generated for a "SUBMITTED" status, effectively formalizing the business logic required to interpret the event log.

Consistency and Anomalies

Let the combined knowledge base be

$$\mathcal{K} \equiv \mathcal{T}_F \cup \mathcal{T}_D \cup \mathcal{T}_{\text{data}} \cup \mathcal{A}_D.$$

The consistency of $\mathcal{K}$ is not guaranteed; in particular, it may be the case that $\mathcal{K} \models \perp$. However, inconsistency is not treated as a strict failure condition. Instead, it provides a diagnostic mechanism for identifying mismatches between empirical observations, their interpretation, and the ontological commitments under which they are analysed. Assuming that the foundational and domain ontologies ($\mathcal{T}_F \cup \mathcal{T}_D$) serve as fixed reference models for the analysis, contradictions indicate that either the observations or their interpretation require further examination.

In practice, an inconsistency in $\mathcal{K}$ can have two primary interpretations:

1. **Source data anomaly:** The observations recorded in $\mathcal{D}_{\text{raw}}$ do not conform to the expected domain behaviour encoded by $\mathcal{T}_{\text{data}}$.
2. **Interpretation anomaly:** The assumptions encoded by $\mathcal{T}_{\text{data}}$ do not adequately capture how the source observations *should* be understood.

A source data anomaly occurs when the data theory is considered an accurate interpretation of the relationship between observations and domain concepts, but the observed data violates those expectations. This may result from logging errors, missing information, or genuine deviations from the intended operation of the system. Resolution therefore requires addressing the source data, improving the information system, or identifying the observation as an exceptional case.

An interpretation anomaly occurs when the observations are accepted as an accurate record of what the information system captured, but the semantic assumptions used

to interpret those observations are insufficient. Refining the data theory may involve correcting an overly simplistic interpretation or introducing additional contextual knowledge through a more detailed ($\mathcal{T}_{\text{data}}^{(1+)}$) theory. A related example appears in process mining data quality research, where “homonymous label” issues describe cases where the same literal label corresponds to distinct underlying behaviours (Suriadi, Andrews, ter Hofstede & Wynn, 2017). For example, a hospital event log may contain multiple “triage assessment” events that represent performing a triage assessment before discharge and reviewing a previous assessment afterwards. Distinguishing these behaviours requires interpreting the event label in relation to its surrounding context rather than treating the literal as the complete semantic description. In terms of data theories, the anomaly arises because a level-zero interpretation may map the same label to a single activity type, whereas a higher-level $\mathcal{T}_{\text{data}}^{(1+)}$ theory enables expressing the conditions to distinguish the distinct intended behaviours from the under-specified data source.

Alternatively, refinement may be driven by analytical requirements rather than correction. In such cases, the source data may accurately represent the behaviour of the information system, but additional abstractions are required for a particular analytical purpose. The data theory provides a formal mechanism for encoding such interpretative choices, such as resolving timestamp collisions through logical ordering or consolidating rapid system-generated status changes into a single meaningful activity.

Ontology-Based Query Answering and Access as Data Theories

OBDA as a Restricted Level-Zero Data Theory Framed within this model, standard OBDA (Xiao, Calvanese, Kontchakov, Lembo, Poggi, Rosati & Zakharyashev, 2018) and Virtual Knowledge Graph (VKG) (Bagosi, Calvanese, Hardi, Komla-Ebri, Lanti, Rezk, Rodriguez-Muro, Slusnys & Xiao, 2014) frameworks are not separate from data theories, but rather represent restricted and optimized instantiations of level-zero data theories.

OBDA can be formally understood as a system operating under the following constraints:

- The structure $\mathcal{D}_{\text{raw}}$ is a relational database.
- The target logical language is a Description Logic formalism, most typically *SROIQ*.
- The source query language $\mathcal{L}_S$ over the relational schema Σ_D is implemented through SQL queries restricted to conjunctive queries (CQs) or unions of conjunctive queries (UCQs).
- The interpretive machinery consists entirely of a level-zero data theory $\mathcal{T}_{\text{data}}^{(0)}$, restricting the system to computing $\emptyset$ -definable sets.

By restricting interpretations to $\emptyset$ -definable (null-definable) sets, OBDA relies on mapping formulas that do not contain explicit parameters from the data source. Instead, these mappings extract structural tuples through source queries Φ_S and map them to assertions over an ontology vocabulary Ψ_F .

R2RML (Dimou, Vander Sande, Colpaert, Verborgh, Mannens & Van de Walle, 2014) natively employs IRI templates, which can be understood as Skolem functions $f(\mathbf{x})$ that generate globally unique entities from relational identifiers. Since no specific constants from the data source are referenced within the mapping axioms, this level-zero interpretation applies uniformly across the source schema. Consequently, OBDA translates the structural organization of the data rather than interpreting the semantic implications of particular data values.

In the case of virtual knowledge graphs, such as Ontop (Bagosi, Calvanese, Hardi, Komla-Ebri, Lanti, Rezk, Rodriguez-Muro, Slusnys & Xiao, 2014), the reliance on virtualization and query rewriting limits their ability to directly support higher-level data theories $\mathcal{T}_{\text{data}}^{(1+)}$. As discussed previously, bridging the semantic gap between source observations and real-world behaviour often requires mappings that depend on the interpretation of specific data values. This moves beyond $\emptyset$ -definable sets, where mappings are based only on schema structure, toward C -definable sets, where mappings may be conditioned on constants from the active domain C .

Specifying such parameter-rich interpretations requires reasoning over concrete data values and their relationships within the data source. This violates the core virtualization assumption, where source data remains passive and is accessed only during query execution. While the strict $\emptyset$ -definable scope of OBDA enables significant performance advantages through query rewriting and other optimizations beyond the scope of this paper, it inherently limits the ability to express highly contextualized, value-dependent interpretations over subsets of the data.

Application of Data Theories with an existing Foundational Ontology

Now having an explicit formalization of data theories and how they relate to some source foundational and/or domain ontologies, we will closely examine two scenarios in which implicit data theories are applied, drawing out the ontological commitments made in each case. Subsequently, we will utilize an existing foundational ontology to formalize the interpretative choices made on the data into data theories, and examine the consequences of alternative choices. This serves as a means of validating and demonstrating data theories in real-world scenarios.

Ontological Commitments in Process Mining Analysis

To examine how data theories can be applied in practice, we consider two real-world cases in which semantic heterogeneity and ambiguity require tacit domain knowledge to interpret the available data. We draw these cases from the Business Process Intelligence Challenges (Lopes & Ferreira, 2019; Thiede, Fuerstenau & Bezerra Barquet, 2018), which provide a particularly suitable setting for this investigation. Each challenge provides a complete dataset, a set of questions posed by the data owner, and multiple solution papers describing how participants interpreted and analysed the data to answer those questions. These solutions employ a range of tools, data-cleaning techniques, and

interpretative assumptions. Because the challenges are primarily intended to produce practical analytical results, these assumptions are often only partially documented, and some interpretations remain implicit or ambiguous.

This informality is a useful property for our purposes rather than a limitation. The solutions capture the kinds of pragmatic interpretative decisions that analysts make when working with real-world data, providing an empirical account of the status quo rather than an idealized or fully explicit semantic specification. We use these accounts as a basis for identifying and formalizing the underlying interpretative assumptions as data theories, allowing us to examine whether those assumptions can be made explicit, logically characterized, and used to reproduce or validate the resulting analyses.

Hand-offs and System Queues within IT Incitement Management Firstly, we focus on an application originating from the IT incident management process at Volvo IT Belgium, the subject of the BPI 2013 challenge (Steeman, 2013). In this challenge, the data is an event log which depicts the status of IT incidents (like an employee reporting that they cannot log in to their laptop, for example) and how they change over time. Table 2 shows an excerpt of the log where the ticket status is recorded over time, starting from an initial Accepted – In Progress status.

Ticket Number	Involved Support Team	Status	Sub Status	Change Date & Time
1-739302563	D2	Accepted	In Progress	2012-05-03 10:46
1-739302563	D2	Accepted	Wait - User	2012-05-03 11:03
1-739302563	D4	Queued	Awaiting Assignment	2012-05-03 11:32
1-739302563	D4	Accepted	In Progress	2012-05-03 11:40
1-739302563	D2	Queued	Awaiting Assignment	2012-05-03 11:42
1-739302563	D2	Accepted	In Progress	2012-05-03 12:17

Table 2. Excerpt of the BPIC 2013 event log depicting the status of incident management tickets

The columns here are the ticket number to identify an occurrence of the ticket resolution process, the involved support team to show which team is involved in the logged status, the status values, indicated by a higher-level status, and lower-level sub status, and the timestamp to indicate when the ticket status was logged.

The business questions largely concerned hand-offs between support teams, particularly whether tickets could be resolved by the initial “first-line” handler rather than being escalated to second- or third-line support. This reflects a “push to front” principle: issues should remain with the initial support team wherever possible, minimizing unnecessary internal escalation. Accurately identifying hand-offs is therefore central to determining how often work is transferred between support levels and where such escalation occurs.

A key challenge in identifying hand-offs is the mismatch between a uniform interpretation of the recorded data and the behaviour it represents in the real world. The *Queued* status provides a simple example. In this information system, handing a ticket from Bob to Alice is a two-stage process: Bob first places the ticket in Alice’s queue,

after which Alice must accept it before taking control. The event log, however, does not explicitly distinguish these two stages.

Consider the simplified example in Table 3. A naive interpretation based only on the schema would treat the support person recorded on the *Queued* event as having begun participating at 11:00. This incorrectly places the hand-off at 11:00 and attributes control of the ticket to Alice from that point. In reality, the 11:00 event records only Bob's *attempt* to hand the ticket to Alice; Alice does not assume control until she accepts it at 11:30. Correctly interpreting this distinction therefore requires knowledge of the semantics of the *Queued* value, rather than treating each recorded support person as having participated whenever they appear in the log.

Involved Support Person	Status	Change Date & Time
Bob 1st Line	Accepted	2012-05-05 10:45
Alice 2nd Line	Queued	2012-05-05 11:00
Alice 2nd Line	Accepted	2012-05-05 11:30

Table 3. Simplified log to demonstrate queued status

This is clearly a relevant distinction to be made when determining the timings and durations of control and hand-offs, but an even more critical distinction can be made in the case of queued tickets that fail to be accepted. Take for example the simplified log snippet in Table 4, which exemplifies an issue encountered in the real-world log.

Involved Support Person	Status	Change Date & Time
Bob 1st Line	Accepted	2012-05-05 10:45
Alice 2nd Line	Queued	2012-05-05 11:00
Bob 1st Line	Accepted	2012-05-05 11:30

Table 4. Simplified log to demonstrate non-accepted queued statuses

In this log, the same naive schema-based interpretation of the involved support person column seems to indicate two hand-offs: firstly from Bob to Alice, and another from Alice back to Bob. However, the real-world interpretation in light of the knowledge of how the queuing system works reveals a different interpretation. In light of this domain knowledge, Alice is never involved and instead this data depicts a situation where Bob attempts to hand off the ticket to Alice via the system queue, but Alice does not accept and instead the ticket control goes back to Bob. Thus the naive interpretation here would result in a false positive; a more detailed interpretation is required in order to correctly identify the real-world behaviour implied by the observations within the log. Furthermore, this interpretation relies on domain knowledge that does not come from the dataset itself, a foundational ontology, or from a domain ontology. While a domain ontology may describe status values and the distinction of a system queue, it does not explicitly link the *Queued* constant in the data to this knowledge. Especially

given the importance of the push-to-front mechanism and detecting and analyzing those behaviours, it is critical to correctly identify and interpret this behaviour (Kang, Kang, Lee, Noh, Kim, Lim, Kim & Hong, 2013).

Loan Application Outcomes For the second case, we focus on the data from a dutch financial institute (Povalyaeva, Khamitov & Fomenko, 2017) (which will from here on be referred to as “the bank”) that is the focus in the 2017 BPI challenge. In this application, the scenario is a loan application process. An example of the event log upon which this scenario is based is given in Table 5.

Application ID	Activity	Transition	Timestamp
Application_3092	Create Application	complete	2016-01-16 09:15
Application_3092	A_submitted	complete	2017-01-16 10:15
Application_3092	W_handle leads	schedule	2017-01-16 10:17
Application_3092	W_validate application	start	2017-01-16 14:25
Application_3092	A_validating	complete	2017-01-16 14:25
Application_3092	W_call incomplete files	complete	2017-01-16 15:20
Application_3092	O_Cancelled	complete	2017-01-18 11:04

Table 5. Excerpt of the BPIC 2017 event log depicting the activity of a loan application

In this instance, the process owner is interested in how incomplete loan applications relate to their eventual outcome. Applications with missing or defective information may be returned to the customer for revision, and the bank hypothesizes that more frequent requests for additional information are associated with lower acceptance rates.

Answering this question requires several interpretative decisions that are not defined by the event log itself. In particular, “outcome” must be defined from the observed events: one might treat the final event as the outcome, or designate particular events such as *Accepted* and *Rejected* as outcome events. Similarly, a measure of the frequency of incompleteness must be defined to compare traces. Here, we adopt the application outcome classification from a solution paper for this challenge (Povalyaeva, Khamitov & Fomenko, 2017), which classifies traces according to the presence or absence of particular events, as shown in Table 6. This provides a concrete interpretation of the business question that can be formalized as a data theory.

This definition of outcome is deliberately pragmatic: it specifies which events classify an application as complete or incomplete without addressing the underlying ontological meaning of these states. It does not characterize the application as an entity, its participation in a process, or the properties that distinguish an ongoing from a completed application. Instead, domain knowledge is implicitly encoded in a data-driven criterion based on the events observed in a trace.

This leaves room to formalize the interpretation as a data theory. From the practitioner’s perspective, the primary concern is obtaining a useful answer and specifying how it can be derived from the available data; the ontological commitments underlying that analysis need not be explicit. An ontology-driven system can preserve

Application Outcome	Definition
Application in progress	No observed A_Cancelled, A_Pending, or A_Denied events.
Missing client response	No observed O_Returned events.
Client declined offer	Observed O_Returned and A_Cancelled events.
Client declined offer after request for incomplete files	Observed O_Returned, A_Cancelled, and W_Call_incomplete_files events.
Bank declined the application	Observed A_Denied event.
Successfully accepted application	Observed A_Pending event.

Table 6. Defined application outcomes for the BPIC 2017 event log.

this pragmatic interface while formally representing those commitments through the data theory, allowing the tooling to make explicit what remains implicit in the analytical specification. The implementation of this approach, including how data theories can remain concealed from practitioners when appropriate, is discussed in Section .

Creating Data Theories with PSL

Now with an account of how two distinct real-world applications utilize external knowledge to interpret data, we will formalize these interpretations with data theories. For this task, we will ground the data theories in an existing foundational ontology of processes, namely the process specification language (PSL) (Grüninger, 2004).

In order to accurately present the data theories constructed within this section, we will firstly provide some brief background on the key concepts contained within PSL as they will be utilized throughout this demonstration and validation. For a more thorough application-oriented introduction to PSL, we refer the reader to (Grüninger, 2009). We also make the important acknowledgement that while PSL is chosen to demonstrate the theory within this paper, the theory itself is specifically made to be ontology-agnostic. The ontological commitments contained within each of these analysis scenarios may be interpreted differently given an alternative foundational ontology to be extended, and even with the same foundational ontology, could be adapted differently, while all being equally valid data theories.

The Process Specification Language The Process Specification Language (PSL) is an ontology designed to facilitate the unambiguous exchange of process information among software applications. PSL consists of a set of first-order logic theories organized into modules. The majority of these modules are related to one another by conservative extensions.

The foundational module is **PSL-Core** (T_{psl_core}), and all other core theories within the ontology operate as consistent extensions of this foundation. Together, PSL-Core and its core theory extensions axiomatize the fundamental concepts within the ontology:

- **Activities:** Repeatable patterns of behaviour. In natural language, they may be commonly interchanged with a process or a task; for example, “the process of applying for a loan” or “the task of baking a pizza” would be activities in PSL. For example, one may describe the class of activities in which a lender l loans an amount x to a borrower b with the term $loan(l, b, x)$.
- **Activity Occurrences:** The temporally concrete realizations of activities. For example, “Bob applying for a home renovation loan last Tuesday” would be an occurrence of the *loan application* activity. Unlike other approaches to modelling processes, activity occurrences in PSL are not instances of activities, since activities are not themselves classes in PSL, but instances of the activity class.
- **Timepoints:** A linearly ordered class represented by the extension of the *before* relation. Activity occurrences are associated with a single begin timepoint and a single end timepoint, with $begin \leq end$.
- **Objects:** The class of things which are not activities, activity occurrences, or timepoints, and are most associated with the notion of continuants. Objects are most typically utilized to describe processes by way of the *participates.in*(x, o, t) relation. For example, the chef who participates in an occurrence of baking a pizza on Friday at 10am.
- **Fluents:** Properties in the domain that may change over time. Axiomatized in the discrete state core theory ($T_{disc.state}$), fluents are an important concept for representing dynamic behaviour, where a change in state is captured by the set of fluents that are either achieved or falsified by an activity occurrence. For example, $holds(under_review(a), o_r)$ represents that application a is currently under review immediately after the occurrence o_r .

Hand-offs and system queues For the BPIC 2013 incident management dataset, the interpretation of the event log with respect to the Process Specification Language (PSL) leverages the foundational concepts of *fluents* and participation. Event records in the raw log correspond to observations of fluents at particular timepoints and therefore map most directly to the PSL predicate holds.

Consider a raw dataset $\mathcal{D}_{raw}$ consisting of a relational table, IncidentLog, recording status changes of support tickets. To specify the data theory, we first examine one row from the source table:

Ticket ID	Involved Support Person	Status	Change Date & Time
Ticket1	Bob 1st Line	Accepted	2012-05-05 10:45

Table 7. Example record from the BPIC 2013 incident log.

To operationalize this data, the level-zero data theory $\mathcal{T}_{data}^{(0)}$ defines an interpretation of the source schema Σ_S with respect to the PSL foundational ontology $\mathcal{T}_F$. The

interpretation is specified by the following mapping rule:

$$\begin{aligned} \forall id, p, s, t \text{ IncidentLog}(id, p, s, t) \rightarrow \\ \text{holds}(f_{\text{status}}(id, p, s), f_t(t)) \wedge \text{participates}(f_{sp}(p), f_{occ}(id, p, s, t), f_t(t)) \end{aligned} \quad (1)$$

Here, $\Phi_S(\mathbf{x})$ denotes the query over the `IncidentLog` relation. The Skolem function $f_{\text{status}}(id, p, s)$ generates a unique fluent individual representing the status of a particular ticket for a particular support person. Likewise, $f_{occ}(id, p, s, t)$ generates a unique occurrence individual, $f_{sp}(p)$ maps a raw resource identifier to a support person individual, and $f_t(t)$ maps a raw timestamp to a PSL timepoint.

The distinction between the source event and the PSL occurrence is important here. The event recorded in the log is an atomic observation: it has no direct counterpart in PSL. When that observation is interpreted through PSL, however, the ontology requires the corresponding fluent to be associated with an occurrence that achieves or falsifies it, or for which it serves as a precondition. The data theory therefore introduces an occurrence individual that is not explicitly present in the source data. Intuitively, the log tells us that a ticket has become accepted; interpreting this observation in PSL requires us to posit an occurrence through which that change in state came about. The occurrence is therefore anonymous in the sense that the source data provides no identifier for it, even though it is required by the ontological interpretation.

Applying the level-zero data theory to the tuple

$$c = (\text{Ticket1}, \text{Bob 1st Line}, \text{Accepted}, 2012-05-05 \text{ 10:45})$$

produces the data assertions $\mathcal{A}_D$. For readability, we replace the Skolem term $f_{\text{status}}(\text{Ticket1}, \text{Bob 1st Line}, \text{Accepted})$ with the domain-specific fluent *accepted*(*Bob*, *Ticket1*), map $f_{sp}(\text{Bob 1st Line})$ to the individual *Bob*, and substitute the generated occurrence $f_{occ}(c)$ with the identifier *occ₁*, in line with the naming conventions of PSL.

The resulting assertions include:

$$\begin{aligned} \text{holds}(\text{accepted}(\text{Bob}, \text{Ticket1}), 2012-05-05 \text{ 10:45}) \in \mathcal{A}_D, \\ \text{participates}(\text{Bob}, \text{occ}_1, 2012-05-05 \text{ 10:45}) \in \mathcal{A}_D. \end{aligned}$$

This interpretation establishes the structural correspondence between the raw `IncidentLog` record and its intended semantics within our chosen foundational ontology, PSL.

Applying this same level-zero data theory to a log entry with status `Queued`—such as Alice’s entry at 11:00 in Table 4—produces the following assertions:

$$\begin{aligned} \text{holds}(\text{queued}(\text{Alice}, \text{Ticket}_1), t) \in \mathcal{A}_D, \\ \text{participates}(\text{Alice}, \text{occ}_2, t) \in \mathcal{A}_D. \end{aligned}$$

In this case, the second assertion does not capture the intended semantics of the underlying process. From our domain knowledge, we know the *queued* status records that another support person (in this case, Bob) has placed the ticket into Alice’s queue;

it does not entail participation in any occurrence by Alice at this point in time. Thus the mapping given by $\mathcal{T}_{\text{data}}^{(0)}$ is too coarse.

This motivates the introduction of a higher-level data theory, $\mathcal{T}_{\text{data}}^{(1+)}$, which refines the structural interpretation by supplementing the mapping with the active domain of constants, $C = \text{const}(\mathcal{D}_{\text{raw}})$. By incorporating dataset values, such as the literal string `Queued` $\in C$, the data theory moves beyond purely schema-level interpretations and allows the generation of C -definable assertions. Rather than applying a single interpretation to every status value, the theory is able to introduce distinct mappings for queued and non-queued observations.

For the `Queued` case, this more detailed data theory therefore replaces the generic level-zero mapping with the following mapping:

$$\forall id, p, t \left(\text{IncidentLog}(id, p, \text{Queued}, t) \rightarrow \text{holds}(\text{queued}(f_{sp}(p), id), f_t(t)) \right). \quad (2)$$

Under this interpretation, no participation literal is generated for the target support person. For analyses concerned specifically with resource participation, this omission is sufficient, as it prevents the erroneous inference that Alice was actively involved in the queueing occurrence. Note that because the logic operates under the open-world assumption, this mapping rule does not imply that Alice's participation in this occurrence is inconsistent with our data theory, but we may wish to add domain closure axioms such that is inconsistent.

However, alternative interpretations may be appropriate depending on the analytical objective. If the goal is to capture that a hand-off *attempt* occurred, or to explicitly represent the system queue as an entity involved in the event, the mapping can be extended accordingly. For example, the theory may introduce a *systemQueue* object that participates in the occurrence rather than the target resource:

$$\begin{aligned} \forall id, p, t \left(\text{IncidentLog}(id, p, \text{Queued}, t) \rightarrow \text{holds}(\text{queued}(f_{sp}(p), id), f_t(t)) \right. \\ \left. \wedge \text{participates}(\text{systemQueue}, f_{occ}(id, p, \text{Queued}, t), f_t(t)) \right). \quad (3) \end{aligned}$$

Thus, this higher-level data theory not only corrects a naive and simplistic interpretation; it makes explicit the ontological commitments of our particular analytical regime. Whether a queued observation represents no participation, participation by a system resource, or a more complex hand-off event is determined by the intended interpretation explicitly encoded in the data theory.

Loan Application Outcomes For the BPIC 2017 loan application case, we use PSL subactivity occurrences to characterize traces by their outcomes. Outcome is determined by the observations and activities recorded within each trace. We interpret each trace (alternatively also called case, in process mining terminology) as a subactivity occurrence of a top-level loan application process, providing the ontological context in which these observations are interpreted.

One level down from here, individual events recorded in the log are interpreted as occurrences of subactivities that compose the top-level case. By detecting the

presence of specific terminal subactivity occurrences—such as an acceptance, denial, or cancellation—we can formally infer the outcome of each loan application trace.

Crucially, this demonstrates that a data theory extends beyond just a structural mapping; in the context of processes, it actively participates in specifying *process descriptions*. The formal classification of a trace based on its constituent subactivities is an operational interpretation required for a specific analytical regime, rather than an axiom belonging to a domain ontology. Therefore, these classifying process descriptions belong strictly within the higher-level data theory $\mathcal{T}_{\text{data}}^{(1+)}$.

Consider a raw event log relation, `EventLog(case_id, activity, timestamp)`. A level-zero data theory $\mathcal{T}_{\text{data}}^{(0)}$ establishes the baseline activity by uniformly asserting that every trace captured in the log is an occurrence of the overarching top-level process:

$$\forall c, a, t \text{ EventLog}(c, a, t) \rightarrow \text{occurrence_of}(f_{\text{trace}}(c), P) \quad (4)$$

Where $f_{\text{trace}}(c)$ acts as a globally scoped Skolem function generating the occurrence representing the given loan application case identified by c , and P is the top-level process.

To characterize traces by their outcomes, we must specialize these occurrences based on the specific activity labels present in the data. This requires $\mathcal{T}_{\text{data}}^{(1+)}$, which first generates C -definable sets by conditioning mapping rules on specific constants (e.g., the string literal `A_Denied`):

$$\begin{aligned} \forall c, t \left(\text{EventLog}(c, \text{A_Denied}, t) \rightarrow \right. \\ \left. \text{occurrence_of}(f_{\text{occ}}(c, \text{A_Denied}, t), \text{DenialActivity}) \right. \\ \left. \wedge \text{isOccuringAt}(f_{\text{occ}}(c, \text{A_Denied}, t), f_t(t)) \right). \end{aligned} \quad (5)$$

Following standard PSL conventions for process descriptions, activities are characterized by the subactivity occurrences they entail. To classify our traces, we introduce a process description into the data theory that mirrors this structure. By formulating it as an equivalence, we allow the logical evaluation to classify any overarching occurrence o that contains a *DenialActivity* as an occurrence of a *DeniedApplicationProcess*:

$$\begin{aligned} \forall o \left(\text{occurrence_of}(o, \text{DeniedApplicationProcess}) \right. \\ \equiv \\ \left. \exists o_1 \left(\text{subactivity_occurrence}(o_1, o) \wedge \right. \right. \\ \left. \left. \text{occurrence_of}(o_1, \text{DenialActivity}) \right) \right). \end{aligned} \quad (6)$$

By embedding these process descriptions directly into the data theory, we formally link the C -definable observations generated from the raw data (the mapped `A_Denied` subactivities) to their analytical outcomes. When the data theory is applied to the log, the occurrence literal generated by $f_{\text{trace}}(c)$ satisfies this process description definition,

successfully classifying the overarching trace. This further demonstrates the utility of a higher-level data theory as an expressive interpretive layer, capable of constructing the necessary formalisms to bridge raw observations and sophisticated reasoning goals.

Implementation and Pragmatics

In line with the overarching goal of rendering applied ontologies more data-driven and operationally realizable, this work extends beyond formal theoretical specification. We provide an open-source implementation designed for seamless integration with real-world, large-scale datasets [‡]. This Ontology-driven Process Mining Analysis Library (OPAL) builds upon the architecture for operational realization in previous work (Moher & Gruninger, 2025b), wherein data theories are operationalized through logical wrappers embedded directly within standard data science workflow tools, predominantly Python.

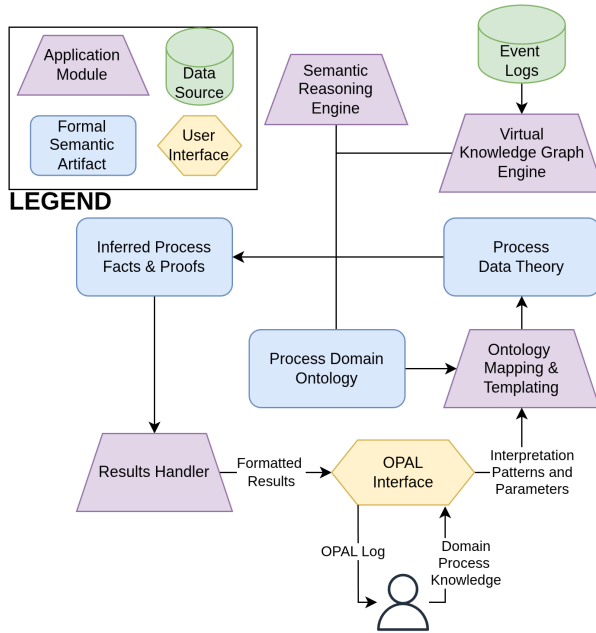


Figure 2. High-level architecture of OPAL

While the primary focus of this paper remains the formal specification of data theories, bridging the gap between theoretical semantics and pragmatic implementation over large event logs necessitates novel implementation considerations. To address the inherent trade-off between computational tractability and the need for highly expressive process reasoning, OPAL employs a *hybrid reasoning architecture*.

[‡]The implementation is available at <https://github.com/Ontology-Benchmarks/opal>

The motivation for this hybrid approach is twofold. On one hand, standard Semantic Web ontologies and pure Ontology-Based Data Access (OBDA) approaches lack the logical expressivity required for deep, complex process logic (Katsumi & Grüninger, 2015; Botti Benevides, Bourguet, Guizzardi, Peñaloza & Almeida, 2019). On the other hand, encoding the entirety of the domain logic in First-Order Logic (FOL) and relying on automated theorem provers to execute all inferences over massive datasets quickly becomes computationally intractable. During our development lifecycle, we evaluated several mature reasoning systems—including Vampyre (Blei, Harrelson, Jhala, Necula, Rahul, Weimer & Weitz,) for first-order resolution, Z3 (De Moura & Bjørner, 2008) for SMT and fixed point logic, and several Prolog (Clocksin & Mellish, 2003) environments. However, we encountered performance and usability bottlenecks when applying these methods to the real-world datasets typical of process mining.

To resolve these issues, we adapt and extend OBDA virtualization (Botoeva, Calvanese, Cogrel, Rezk & Xiao, 2016) through a novel strategy of *selective materialization*. Under this paradigm, OPAL leverages the mature and highly scalable Semantic Web-based mapping tools to perform efficient data virtualization for the vast majority of lightweight, structural mappings. This keeps memory overhead low and capitalizes on the efficiency of underlying relational database engines (in this case the highly efficient duckDB (Raasveldt & Mühleisen, 2019). Conversely, where heavier reasoning is required, minimal contextually relevant subsets of the mapped data assertions are dynamically extracted and delegated to Soufflé (Scholz, Jordan, Subotić & Westmann, 2016) alongside a datalog-encoded version of relevant PSL axioms. Soufflé, a highly optimized Datalog reasoning engine, is exceptionally well-suited for these inference tasks due to its bottom-up evaluation strategy and translation of Datalog to optimized C++ code.

Using this hybrid architecture, the system materializes these complex results as semantically rich, domain-specific inferred facts informed by foundational and/or domain ontologies. Also pragmatically useful is OPAL’s output of a metadata artifact to record the data theory applied. By explicitly capturing all mappings, ontological templates, and interpretive assumptions, this artifact guarantees full provenance, reproducibility, and verifiability of the data pipeline, regardless of the target domain.

In the interest of accessibility and promoting wider adoption amongst practitioners without a knowledge representation background, the system operationalizes data theories through a parametric, pattern-based approach, in line with our previous work (Moher & Gruninger, 2025a). Recognizing that domain practitioners often lack the specialized expertise required to construct and interpret complex logical axioms, OPAL encapsulates complex formal semantics as reusable axiom patterns, strictly grounded in a foundational or domain ontology. As illustrated by the flow of “Interpretation Patterns and Parameters” into the OPAL Interface in Figure 2, users simply instantiate these templates, just as they would the methods of a python library, to generate a full-fledged data theory. This paradigm empowers practitioners to slice and interpret their data by making intuitive, real-world ontological commitments rather than being forced to learn the syntax of various reasoning systems. For example, when specifying the outcomes of a given process, a user can classify a distinct “variant” simply by defining a parameter that

requires the occurrence of a specific milestone event. Instead of manually engineering complex, purely analytical data filters, the user declares their structural assumptions, and the system automatically compiles them into logical sentences and mappings in the data theory. Ultimately, this parametric nature bridges the gap between practical domain knowledge and formal semantics.

As illustrated in Figure 2, this design enforces a strict modular separation between the formal specification of domain knowledge and the underlying execution mechanisms. By formally encoding interpretive decisions as data theories and executing them through a robust, highly scalable architecture, we provide the applied ontology community with a working implementation capable of driving real-world impact. This approach demonstrates that explicit, formally verified semantics—rather than opaque analytical assumptions or hard-coded pipeline scripts—can successfully and practically drive large-scale data analysis across diverse application domains.

Discussion and Future Work

The introduction of data theories enforces an important push for applied ontology from largely representational applications towards the grounded and data-driven goal of *operational realization* (Moher & Gruninger, 2025b). By explicitly formalizing the interpretive layer between raw data sources and foundational ontologies, data theories expose domain knowledge that is traditionally embedded within ad-hoc preprocessing scripts or left implicit in analytical practice.

This formalization also provides a principled basis for future ontology evaluation strategies. Assessing the operational realization of an ontology across diverse domains is challenging because real-world data is inherently noisy, heterogeneous, and incomplete. Data theories offer a transparent and measurable basis for this evaluation by making the required interpretive assumptions explicit. The complexity, size, and logical consistency of a data theory required to operationalize an ontology provide objective indicators of its practical applicability. An ontology that requires an excessively complex data theory to interpret simple domain phenomena may be overly abstract, whereas one that cannot capture essential distinctions without introducing inconsistencies may lack sufficient expressive power.

Beyond evaluating individual applications, this architecture provides the foundation for a process ontology benchmarking environment. Rather than assessing ontologies using only competency questions (Gruninger & Fox, 1995) or case studies based on conceptual and referential qualities (El Ghosh, Naja, Abdulrab & Khalil, 2025; Ondraszek, Waitelonis, Keller, Niessner, Jacyszyn & Sack, 2025), benchmarking can be grounded in curated collections of real-world challenge problems spanning multiple domains, datasets, and process characteristics. The proposed architecture provides a common methodology for operationalizing candidate ontologies against these benchmark problems, allowing comparisons to be performed under consistent interpretive assumptions. Benchmark problems may further be categorized according to the reasoning tasks they exercise, such as classification, consistency checking, temporal reasoning, trace completeness, or event abstraction. Such an environment would enable

systematic, data-driven evaluation of the operational complexity, expressiveness, and practical utility of competing process ontologies.

These advantages are particularly evident in process-centric applications. Process mining depends upon sequential, temporal, and causal interpretations that are highly sensitive to how data is understood. Distinguishing, for example, between an active queuing event and a passive queued state is not a simple pre-processing step but an ontological commitment. Data theories make these commitments explicit, enabling process interpretations that are semantically grounded, reproducible, and formally verifiable. More broadly, although operational realization and ontology benchmarking are applicable beyond the process domain, processes provide an especially suitable starting point because of their prescriptive nature, rich temporal structure, and mature analytical methodologies (Van Der Aalst, 2012). The recurring patterns of interpretation found in process analysis make the domain particularly well suited to the reusable reasoning patterns captured by data theories.

While the proposed framework and its implementation in the Ontology-driven Process Mining Analysis Library (OPAL) demonstrate the feasibility of this approach, several avenues remain for future research.

Object-Centric Event Logs

Traditional process mining represents process executions using a single case identifier, often obscuring the many-to-many relationships that exist between real-world entities. Although the Object-Centric Event Log (OCEL) standard (Ghahfarokhi, Park, Berti & van der Aalst, 2021; Berti, Koren, Adams, Park, Knopp, Graves, Rafiei, Liß, Unterberg, Zhang & others, 2024) addresses this limitation by allowing multiple objects to participate in each event, it faces similar issues to the existing case-centric standard in terms of flexibility and semantic heterogeneity.

Future work will extend the data theory framework to natively support object-centric process mining. By incorporating explicit ontological representations of multi-object interactions, object lifecycles, and part-whole relationships, data theories can provide a more rigorous semantic grounding, enabling reasoning over complex enterprise processes.

Improving OPAL Accessibility

Although OPAL's parametric, pattern-based architecture (Moher & Gruninger, 2025a) substantially reduces the effort required to construct data theories, further improvements are needed to make the framework accessible to non-experts. Future work will expand the library of reusable ontological patterns and investigate more intuitive authoring environments, including natural language interfaces capable of translating domain-specific descriptions into formal logical specifications. These developments would also naturally support the benchmarking environment by allowing benchmark challenges and data theories to be authored and shared more easily across the community.

Integration with Process Mining Ecosystems

OPAL currently functions as a standalone semantic reasoning layer. A natural next step is tighter integration with established process mining frameworks (Van Der Aalst, 2012) and enterprise process mining platforms. Providing native support for semantically enriched assertions ($\mathcal{A}_D$) and hybrid reasoning services would enable existing process mining algorithms to operate directly over ontology-grounded representations, supporting semantically informed process discovery, conformance checking, and process analysis without requiring practitioners to leave their existing analytical environments.

Ultimately, these developments aim to establish ontology-aware process mining as both a practical analytical methodology and a rigorous evaluation framework. For the applied ontology community, this encourages the development of process ontologies grounded in operational requirements rather than solely conceptual adequacy. For the process mining community, it provides a principled foundation for integrating formal ontologies into existing workflows, improving the transparency, verifiability, reproducibility, and comparability of ontology-aware process analyses.

Conclusion

In this paper, we provided a novel approach to a foundational problem in applied ontology: the semantic gap between real-world phenomena and the raw data that records them. While the field has constructed rich, high-level models to represent reality, it has historically lacked formal, standardized mechanisms to integrate these models with real-world datasets. To bridge this gap, we introduced data theories—a formal interpretive layer that explicitly defines how raw data observations are interpreted into meaningful ontological concepts through a foundational ontology.

By focusing on the domain of processes and utilizing process mining as our empirical basis, we demonstrated how data theories are used to resolve real-world data interpretation decisions. We formalized the concept of a data theory across two distinct levels: a level-zero theory ($\mathcal{T}_{\text{data}}^{(0)}$) for baseline structural schema alignment, and higher-level theories ($\mathcal{T}_{\text{data}}^{(1+)}$) for context-aware, parameter-driven interpretation. Through this formalization, we established how existing data access paradigms, such as OBDA and OMQA (Lenzerini, 2011; Gagnon, 2007; Xiao, Calvanese, Kontchakov, Lembo, Poggi, Rosati & Zakharyashev, 2018), function not as separate methodologies, but as restricted, level-zero data theories.

Furthermore, we provided a concrete, open-source implementation (OPAL) that validates this methodology using real-world event logs. This hybrid reasoning architecture successfully separates the basic structural mapping of a database from the nuanced, context-aware rules necessary to truly understand the behaviour the data represents. By doing so, it allows practitioners to execute scalable semantic interpretation without sacrificing logical expressivity or computational tractability.

Ultimately, data theories enable a structured and logically-grounded means to enable foundational ontologies to be genuinely useful in data-driven scenarios. By providing a rigorous yet pragmatic mechanism for data interpretation, this framework progresses the

goal of enabling applied ontology to achieve greater *operational realization*. In doing so, it ensures that expressive ontologies are equipped to support complex, large-scale data analysis, establishing a foundation for more semantically grounded, transparent, and verifiable enterprise information systems.

References

- Van der Aalst, W. M. (2013). Business process management: a comprehensive survey. *International Scholarly Research Notices*, 2013(1), 507984.
- Accorsi, R., Di Ciccio, C., Jayarathna, L. & Verbeek, H. (2022). Rethinking the input for process mining: Insights from the xes survey and workshop. In *Process Mining Workshops* (p.3).
- Bagosi, T., Calvanese, D., Hardi, J., Komla-Ebri, S., Lanti, D., Rezk, M., Rodriguez-Muro, M., Slusnys, M. & Xiao, G. (2014). The ontop framework for ontology based data access. In *The Semantic Web and Web Science: 8th Chinese Conference, CSWS 2014, Wuhan, China, August 8-12, 2014, Revised Selected Papers 8* (pp. 67–77).
- Bautista, A. D., Wangikar, L. & Akbar, S. M. K. (2012). Process mining-driven optimization of a consumer loan approvals process. *BPI Challenge*.
- Bergami, G., Maggi, F. M., Marrella, A. & Montali, M. (2021). Aligning data-aware declarative process models and event logs. In *Business Process Management: 19th International Conference, BPM 2021, Rome, Italy, September 06–10, 2021, Proceedings 19* (pp. 235–251).
- Berti, A., Koren, I., Adams, J. N., Park, G., Knopp, B., Graves, N., Rafiei, M., Liß, L., Unterberg, L. T. G., Zhang, Y. et al. (2024). Ocel (object-centric event log) 2.0 specification. *arXiv preprint arXiv:2403.01975*.
- Bienvenu, M. (2016). Ontology-mediated query answering: Harnessing knowledge to get more from data. In *IJCAI* (pp. 4058–4061).
- Blei, D., Harrelson, C., Jhala, R., Necula, G. C., Rahul, S. P., Weimer, W. & Weitz, D. The vampyre theorem prover. <https://people.mpi-sws.org/~rupak/Vampyre/>. Accessed: 2024-05-30.
- Borgo, S., Ferrario, R., Gangemi, A., Guarino, N., Masolo, C., Porello, D., Sanfilippo, E. M. & Vieu, L. (2022a). Dolce: A descriptive ontology for linguistic and cognitive engineering. *Applied ontology*, 17(1), 45–69.
- Borgo, S., Galton, A. & Kutz, O. (2022b). Foundational ontologies in action: Understanding foundational ontology through examples.
- Botoeva, E., Calvanese, D., Cogrel, B., Rezk, M. & Xiao, G. (2016). Obda beyond relational dbs: A study for mongodb. In *CEUR Workshop Proceedings*, Volume 1577.

- Botti Benevides, A., Bourguet, J.-R., Guizzardi, G., Peñaloza, R. & Almeida, J. P. A. (2019). Representing a reference foundational ontology of events in sroiq. *Applied Ontology*, 14(3), 293–334.
- Brandão, A. A. & Loureiro, G. (2020). An overview of the bfo-basic formal ontology-and its applicability for satellite systems.
- Bukhsh, Z. A., Saeed, A. & Dijkman, R. M. (2021). Processtransformer: Predictive business process monitoring with transformer network. *arXiv preprint arXiv:2104.00721*.
- Calvanese, D., Kalayci, T. E., Montali, M. & Tinella, S. (2017). Ontology-based data access for extracting event logs from legacy data: the onprom tool and methodology. In *Business Information Systems: 20th International Conference, BIS 2017, Poznan, Poland, June 28–30, 2017, Proceedings 20* (pp. 220–236).
- Clocksin, W. F. & Mellish, C. S. (2003). *Programming in PROLOG*. Springer Science & Business Media.
- De Moura, L. & Bjørner, N. (2008). Z3: An efficient smt solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems* (pp. 337–340).
- Di Ciccio, C., Montali, M. et al. (2022). Declarative process specifications: Reasoning, discovery, monitoring. *Process mining handbook*, 448, 108–152.
- Dimou, A., Vander Sande, M., Colpaert, P., Verborgh, R., Mannens, E. & Van de Walle, R. (2014). Rml: A generic language for integrated rdf mappings of heterogeneous data. *Ldow*, 1184.
- van Dongen, B. F. (2015). Bpi challenge 2015. In *11th International Workshop on Business Process Intelligence (BPI 2015)*.
- El Ghosh, M., Naja, H., Abdulrab, H. & Khalil, M. (2025). Crimonto: A generalized domain-specific ontology for modeling procedural norms of the lebanese criminal law. *Data & Knowledge Engineering*, 158, 102419.
- Fahland, D. (2022). Process mining over multiple behavioral dimensions with event knowledge graphs. In *Process mining handbook* (pp. 274–319). Springer.
- Gagnon, M. (2007). Ontology-based integration of data sources. In *2007 10th International Conference on Information Fusion* (pp. 1–8).
- Ghahfarokhi, A. F., Park, G., Berti, A. & van der Aalst, W. M. (2021). Ocel: A standard for object-centric event logs. In *European Conference on Advances in Databases and Information Systems* (pp. 169–175).

- Grover, V. & Malhotra, M. K. (1997). Business process reengineering: A tutorial on the concept, evolution, method, technology and application. *Journal of operations management*, 15(3), 193–213.
- Grüninger, M. (2004). Ontology of the process specification language. In *Handbook on ontologies* (pp. 575–592). Springer.
- Grüninger, M. (2009). Using the psl ontology. In *Handbook on Ontologies* (pp. 423–443). Springer.
- Grüninger, M. & Fox, M. S. (1995). The role of competency questions in enterprise engineering. In *Benchmarking—Theory and practice* (pp. 22–31). Springer.
- Grüninger, M., Ru, Y. & Thai, J. (2022). Tupper: A top level ontology within standards. *Applied Ontology*, 17(1), 143–165.
- Grüninger, M., Shapiro, S., Fox, M. S. & Weppner, H. (2010). Combining rfid with ontologies to create smart objects. *International journal of production research*, 48(9), 2633–2654.
- Guizzardi, G., Botti Benevides, A., Fonseca, C. M., Porello, D., Almeida, J. P. A. & Prince Sales, T. (2022). Ufo: Unified foundational ontology. *Applied ontology*, 17(1), 167–210.
- Gunther, C. W. & Verbeek, H. (2014). Xes-standard definition.
- Kamath, A. & Das, R. (2018). A survey on semantic parsing. *arXiv preprint arXiv:1812.00978*.
- Kang, C. J., Kang, Y. S., Lee, Y. S., Noh, S., Kim, H. C., Lim, W. C., Kim, J. & Hong, R. (2013). Process mining-based understanding and analysis of volvo it's incident and problem management processes. *BPIC@ BPM*, 85.
- Katsumi, M. & Grüninger, M. (2015). Using psl to extend and evaluate event ontologies. In *International Symposium on Rules and Rule Markup Languages for the Semantic Web* (pp. 225–240).
- Lenzerini, M. (2011). Ontology-based data management. In *Proceedings of the 20th ACM international conference on Information and knowledge management* (pp. 5–6).
- Lopes, I. F. & Ferreira, D. R. (2019). A survey of process mining competitions: the bpi challenges 2011–2018. In *Business Process Management Workshops: BPM 2019 International Workshops, Vienna, Austria, September 1–6, 2019, Revised Selected Papers 17* (pp. 263–274).
- Marrella, A. (2019). Automated planning for business process management. *Journal on Data Semantics*, 8(2), 79–98.

- Moher, R. & Gruninger, M. (2025a). Mining for meaning: Ontology-aware process mining methods through knowledge patterns. In *International Conference on Research Challenges in Information Science*.
- Moher, R. & Gruninger, M. (2025b). Process representation meets operational realization: An architecture for data-driven process ontology application through process mining. In *Formal Ontology in Information Systems* (pp. 196–209).
- González López de Murillas, E., Reijers, H. A. & Van Der Aalst, W. M. (2019). Connecting databases with process mining: a meta model and toolset. *Software & Systems Modeling*, 18(2), 1209–1247.
- Norton, B., Cabral, L. & Nitzsche, J. (2009). Ontology-based translation of business process models. In *2009 Fourth International Conference on Internet and Web Applications and Services* (pp. 481–486).
- Olivares-Alarcos, A., Beßler, D., Khamis, A., Goncalves, P., Habib, M. K., Bermejo-Alonso, J., Barreto, M., Diab, M., Rosell, J., Quintas, J. et al. (2019). A review and comparison of ontology-based approaches to robot autonomy. *The Knowledge Engineering Review*, 34, e29.
- Ondraszek, S. R., Waitelonis, J., Keller, K., Niessner, C., Jacyszyn, A. M. & Sack, H. (2025). Ontologies in motion: A bfo-based approach to knowledge graph construction for motor performance research data in sports science. *arXiv preprint arXiv:2510.15983*.
- Otte, J. N., Beverley, J. & Ruttenberg, A. (2022). Bfo: Basic formal ontology. *Applied ontology*, 17(1), 17–43.
- Pedrinaci, C. & Domingue, J. (2007). Towards an ontology for process monitoring and mining. In *CEUR Workshop Proceedings*, Volume 251 (pp. 76–87).
- Povalyaeva, E., Khamitov, I. & Fomenko, A. (2017). Bpic 2017: density analysis of the interaction with clients. *BPI Challenge*, 2017.
- Raasveldt, M. & Mühleisen, H. (2019). Duckdb: an embeddable analytical database. In *Proceedings of the 2019 international conference on management of data* (pp. 1981–1984).
- dos Santos Garcia, C., Meincheim, A., Junior, E. R. F., Dallagassa, M. R., Sato, D. M. V., Carvalho, D. R., Santos, E. A. P. & Scalabrin, E. E. (2019). Process mining techniques and applications—a systematic mapping study. *Expert Systems with Applications*, 133, 260–295.
- Scholz, B., Jordan, H., Subotić, P. & Westmann, T. (2016). On fast large-scale program analysis in datalog. In *Proceedings of the 25th International Conference on Compiler Construction* (pp. 196–206).

- Smeulders, A. W., Worring, M., Santini, S., Gupta, A. & Jain, R. (2002). Content-based image retrieval at the end of the early years. *IEEE Transactions on pattern analysis and machine intelligence*, 22(12), 1349–1380.
- Sommers, D., Menkovski, V. & Fahland, D. (2021). Process discovery using graph neural networks. In *2021 3rd International Conference on Process Mining (ICPM)* (pp. 40–47).
- Sommers, D., Menkovski, V. & Fahland, D. (2023). Supervised learning of process discovery techniques using graph neural networks. *Information Systems*, 115, 102209.
- Steeman, W. (2013). BPI challenge 2013, incidents.
- Suriadi, S., Andrews, R., ter Hofstede, A. & Wynn, M. T. (2017). Event log imperfection patterns for process mining: Towards a systematic approach to cleaning event logs. *Information systems*, 64, 132–150.
- Swevels, A., Fahland, D. & Montali, M. (2023). Implementing object-centric event data models in event knowledge graphs. In *International Conference on Process Mining* (pp. 431–443).
- Tax, N., Verenich, I., La Rosa, M. & Dumas, M. (2017). Predictive business process monitoring with lstm neural networks. In *International conference on advanced information systems engineering* (pp. 477–492).
- Thiede, M., Fuerstenau, D. & Bezerra Barquet, A. P. (2018). How is process mining technology used by organizations? a systematic literature review of empirical studies. *Business Process Management Journal*, 24(4), 900–922.
- Van Der Aalst, W. (2012). Process mining: Overview and opportunities. *ACM Transactions on Management Information Systems (TMIS)*, 3(2), 1–17.
- Van Der Aalst, W., Adriansyah, A., De Medeiros, A. K. A., Arcieri, F., Baier, T., Blickle, T., Bose, J. C., Van Den Brand, P., Brandtjen, R., Buijs, J. et al. (2012). Process mining manifesto. In *Business Process Management Workshops: BPM 2011 International Workshops, Clermont-Ferrand, France, August 29, 2011, Revised Selected Papers, Part I* 9 (pp. 169–194).
- Xiao, G., Calvanese, D., Kontchakov, R., Lembo, D., Poggi, A., Rosati, R. & Zakharyashev, M. (2018). Ontology-based data access: A survey.